

4.1串的定义和实现

基本概述

串是由零个或多个字符组成的有限序列

串中任意个连续的字符组成的子序列称为该串的子串，包含子串的串相应地称为主串

子串在主串中的位置以子串的第一个字符在主串中的位置来表示

当两个串的长度相等且每个对应位置的字符都相等时，称这两个串是相等的

由一个或多个空格（空格是特殊字符）组成的串称为空格串 其长度为串中空格字符的个数

串的存储结构

定长顺序存储表示

类似于线性表的顺序存储结构，用一组地址连续的存储单元存储串值的字符序列

截断：串的实际长度只能小于等于MAXLEN,超过预定义长度的串值会被舍去

串表示方法

1.用一个额外的变量来存放串的长度

2.串值后面加一个不计入串长的结束标记字符"\0",此时的串长为隐含值

弊端

串值序列的长度超过上界,约定用截断法处理

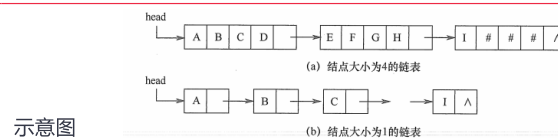
解决方法 用不限定串长的最大长度，即采用动态分配的方式

堆分配存储表示

堆分配存储表示仍然以一组地址连续的存储单元存放串值的字符序列，但它们的存储空间是在程序执行过程中动态分配得到的

块链存储表示

每个结点既可以存放一个字符,也可以存放多个字符。每个结点称为块,整个链表称为块链结构



串的基本操作

StrAssign (T,chars):赋值操作。把串T赋值为chars

StrCopy(&T,S):复制操作。由串S 复制得到串T

StrEmpty (S):判空操作。若S为空串,则返回TRUE,否则返回FALSE

Strcompare ((S,T):比较操作。若S>T,则返回值>0;若 s=T,则返回值=0;若s<T, 则返回值<0

StrLength(S):求串长。返回串S的元素个数

SubString(&Sub,S,pos,len):求子串。用Sub返回串S的第pos个字符起长度为len的子串

Concat(&T,S1,S2):串联接。用T返回由S1和S2联接而成的新串

Index (S,T):定位操作。若主串S中存在与串T值相同的子串,则返回它在主串S中第一次出现的位置;否则函数值为0

clearString(&S):清空操作。将S清为空串

DestroyString(&S):销毁串。将串S销毁

字符集编码

英文字符——ASCII字符集

中英文——Unicode字符集

4.2串的模式匹配

简单的模式匹配算法

- 子串的定位操作通常称为串的模式匹配，它求的是子串(常称模式串)在主串中的位置
- 实现思想
 - 将主串中与模式串长度相同的子串搞出来，挨个与模式串对比
 - 当子串与模式串某个对应字符不匹配时，就立即放弃当前子串，转而检索下一个子串
- 缺点：主串指针会出现回溯现象导致时间开销增加
- 最坏时间复杂度： $O(nm)$,其中n和m分别为主串和模式串的长度。

改进的模式匹配算法—KMP算法

- 字符串的前缀、后缀和部分匹配值
- next 数组：当模式串的第 j 个字符匹配失败时，令模式串跳到 next[j] 再继续匹配
- 获取串的匹配值 'ababa'为例
 - 'a'的前缀和后缀都为空集,最长相等前后缀长度为0
 - 'ab'的前缀为{a},后缀为{b},(a)∩(b)=∅,最长相等前后缀长度为0
 - 'aba'的前缀为{a,ab},后缀为{a,ba},(a,ab)∩(a,ba)={a},最长相等前后缀长度为1
 - 'abab'的前缀{a,ab,aba}∩后缀{b,ab,bab}={ab},最长相等前后缀长度为2
 - 'ababa'的前缀{a,ab,aba,abab}∩后缀{a,ba,aba,baba}={a,aba},公共元素有两个,最长相等前后缀长度为3
- 故字符串'ababa'的部分匹配值为00123
- 当子串和模式串不匹配时，主串指针 i 不回溯，模式串指针 j=next[j]
- 时间复杂度： $O(m+n)$
- 优点：主串不会进行回溯

KMP算法的进一步优化

- KMP算法优化：当子串和模式串不匹配时j=nextval[j] 通过构造nextval函数